



**VNiVERSiDAD
D SALAMANCA**

Facultad de Ciencias
Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

**Desarrollo de una IA para jugar Riichi
Mahjong**

Development of an AI to Play Riichi Mahjong

**Predicción y evaluación estratégica de descartes de Riichi
Mahjong mediante Transformers**

*Prediction and Strategic Evaluation of Riichi Mahjong Discards Using
Transformers*

Autor

Julia Ruiperez de Brito

Tutor

Prof. Vidal Moreno Rodilla

Departamento

Informática y Automática

Salamanca
Junio 2026

Abstract

This project presents an Artificial Intelligence system capable of selecting discard actions in Riichi Mahjong, a traditional Japanese game of imperfect information.

Using a Transformer-based architecture, the model is trained through imitation learning on a dataset composed of real games collected from Tenhou.net. The model is evaluated in terms of predictive accuracy, and the implications of the obtained results are discussed.

In addition to conventional accuracy metrics, this work proposes a complementary evaluation method referred to as *Strategic Accuracy*. It uses categories defined through Shanten and Ukeire to analyse hand efficiency beyond exact agreement with the observed action.

On a reserved evaluation set containing 52,563 game states, the model trained on 500,000 states achieves a top-1 accuracy of 65.64%, a top-3 accuracy of 92.47%, and a strategic accuracy of 94.01%. Compared with the checkpoint identified as 100k, the second model improves all predictive metrics and reduces Shanten-degrading decisions from 5.03% to 1.59%. These results indicate that the model learns relevant discard-selection patterns, although the proposed strategic evaluation remains limited to the aspects represented by Shanten and Ukeire.

Keywords:

Riichi Mahjong, Artificial Intelligence, Transformers, Imitation Learning, Strategic Accuracy, Model Evaluation, Imperfect Information Games.

Resumen

Este trabajo presenta un sistema de Inteligencia Artificial capaz de tomar decisiones de descarte en Riichi Mahjong, un juego tradicional japonés de información imperfecta caracterizado por un elevado nivel de complejidad estratégica.

Para ello, se desarrolla un modelo basado en la arquitectura Transformer entrenado mediante aprendizaje por imitación a partir de un conjunto de datos compuesto por partidas reales obtenidas de la plataforma Tenhou.net. El rendimiento del modelo se evalúa utilizando métricas convencionales de precisión, analizando además las implicaciones de los resultados obtenidos.

Con el objetivo de complementar estas métricas tradicionales, se propone en este trabajo una metodología de evaluación denominada *Precisión Estratégica*, basada en categorías definidas mediante Shanten y Ukeire. Este enfoque permite analizar la eficiencia de las decisiones generadas por el modelo más allá de la coincidencia exacta con la acción observada.

Los resultados experimentales muestran que el modelo entrenado con 500.000 estados alcanza una precisión exacta del 65,64%, una precisión top-3 del 92,47% y una precisión estratégica del 94,01% sobre un conjunto reservado de 52.563 estados. En comparación con el checkpoint identificado como 100k, el segundo modelo mejora todas las métricas predictivas y reduce las decisiones que empeoran el Shanten del 5,03% al 1,59%. Estos resultados indican que el modelo aprende regularidades relevantes para la selección de descartes, aunque la evaluación estratégica queda limitada a los aspectos representados mediante Shanten y Ukeire.

Palabras clave:

Riichi Mahjong, Inteligencia Artificial, Transformers, Aprendizaje por Imitación, Precisión Estratégica, Evaluación de Modelos, Juegos de Información Imperfecta.

Índice general

1	Introducción	7
2	Objetivos	7
3	Conceptos Teóricos	8
3.1	Reglas Básicas	8
3.2	Heurísticas	11
3.2.1	Ukeire	11
3.2.2	Shanten	11
3.3	Arquitectura Transformer	11
3.3.1	Tokens	13
3.3.2	Embeddings	14
3.3.3	Aprendizaje por Imitación	14
4	Estado del Arte	14
4.1	Suphx	14
4.2	Transformers aplicados al Mahjong	14
4.3	Posicionamiento del presente trabajo	15
5	Herramientas y Entorno de Desarrollo	15
6	Desarrollo del Proyecto	16
6.1	Análisis de Exploración de Datos	16
6.2	Distribución de Acciones	17
6.3	Distribución de la Suma de Acciones Válidas de una Sample	18
6.4	Frecuencia de Piezas Descartadas	19
6.5	Representación de un Estado	19
7	Arquitectura del Modelo	22
7.1	Reconstrucción del estado de juego	23
7.2	Tokenización del estado	24
7.3	Capa de embeddings	24
7.4	Transformer Encoder	24
7.5	Cabeza de política y acciones legales	24
7.6	Justificación de las decisiones arquitectónicas	25
7.7	Diseño conceptual de la tokenización	25
7.7.1	Tokens de Mano	26
7.7.2	Tokens de Descarte	26
7.7.3	Tokens de Melds	27
7.7.4	Tokens de Acciones	27
7.7.5	Tokens de Estado de Juego	27
7.7.6	Tokens de Estado de Jugadores	28
7.8	Diseño conceptual del embedding	28
7.8.1	Embedding de Tokens de Piezas	28
7.8.2	Embedding del índice de copia	28
7.8.3	Embedding de Jugador	28
7.8.4	Embeddings Posicionales	29

7.8.5	Embeddings de Metadatos	29
7.8.6	Embedding de Token de Mano	29
7.8.7	Embedding de Token de Descarte	29
7.8.8	Embedding de Token de Meld	30
7.8.9	Embedding de Token de Acción	30
7.8.10	Embedding de Token de Estado de Juego	31
7.8.11	Embedding de Token de Estado de Jugador	31
7.9	Tensorización y embeddings implementados	32
7.10	Configuración final del modelo	32
8	Resultados	32
8.1	Conjunto de evaluación reservado	33
8.2	Métricas predictivas	33
8.3	Evaluación estratégica	34
8.4	Confianza de las predicciones	36
8.5	Coste computacional y eficiencia	37
8.6	Análisis comparativo	37
9	Conclusiones y Líneas de Trabajo Futuro	38
10	Líneas de Trabajo Futuro	38
11	Glosario	40
11.1	Términos de Riichi Mahjong	40
11.2	Términos técnicos en inglés	41

Índice de figuras

1	Posición relativa de los jugadores y vientos en una mesa de Riichi Mahjong.	9
2	Arquitectura Estándar de un Transformer	12
3	Arquitectura BERT (izquierda) frente a arquitectura GPT	13
4	Distribución de Número de Datos del Dataset por tipo de Acciones Válidas	17
5	Distribución del número de acciones válidas del schema de Descarte	18
6	Frecuencia de las Piezas Descartadas	19
7	Flujo de procesamiento y arquitectura del modelo propuesto.	23
8	Comparación de las métricas obtenidas por ambos modelos sobre los 52.563 estados reservados para la evaluación final	34
9	Distribución de las categorías estratégicas de ambos modelos sobre el conjunto de evaluación reservado	35
10	Distribución de la confianza de ambos modelos sobre el conjunto de evaluación reservado	36

Índice de cuadros

1	Objetivos del proyecto	8
2	Categorías de piezas de Riichi Mahjong	9
3	Formas de ganar en Riichi Mahjong	10
4	Acciones posibles en cada turno de Riichi Mahjong	11
5	Tecnologías empleadas durante el desarrollo del proyecto.	16
6	Schemas del Dataset	17
7	Acciones válidas según melds abiertos	18
8	Campos de un Estado de Mahjong	21
9	Campos de un Jugador en un Estado de Mahjong	21
10	Campos de un Meld en un Estado de Mahjong	21
11	Campos de una Acción Válida en un Estado de Mahjong	22
12	Categorías de Informaciones	26
13	Campos de Token de Mano	26
14	Campos de Token de Descarte	26
15	Campos de Token de Melds	27
16	Campos de Token de Acciones	27
17	Campos de Token de Estado de Juego	27
18	Campos de Token de Estado de Jugadores	28
19	Resultados sobre el conjunto de evaluación reservado	33
20	Distribución de categorías estratégicas en el conjunto de evaluación reservado	35
21	Tiempo de cálculo de la evaluación sobre los 52.563 estados reservados	37

1 Introducción

Riichi Mahjong es un juego de información imperfecta, caracterizado por una elevada complejidad estratégica y por la presencia de información oculta. Su espacio de estados se estima en el orden de 10^{48} [10]. Además, el orden de juego puede alterarse, ya que ciertas acciones, como el *pon*, el *chi* o el *kan*, permiten interrumpir la secuencia habitual de turnos. Esto incrementa aún más la complejidad del juego.

Este proyecto desarrolla una Inteligencia Artificial que predice una acción de descarte a partir de la información observable de un estado de juego, que incluye información secuencial y estática. El modelo se basa en la arquitectura Transformer y se entrena mediante aprendizaje por imitación utilizando 500.000 estados de juego reales obtenidos de partidas de la plataforma Tenhou.net.

Durante el desarrollo del proyecto se obtienen diferentes métricas de precisión, analizando tanto la coincidencia con el descarte observado como la precisión estratégica, que se basa en categorías semánticas propias del juego calculadas mediante el *Shanten* y el *Ukeire*.

El esquema general de esta memoria comienza con los objetivos del proyecto, seguido de una sección de teoría, donde se explican el juego de Mahjong y las heurísticas del juego, así como la arquitectura de Transformers y el aprendizaje por imitación. Después, se realiza un repaso del estado del arte, donde se analizan los trabajos relacionados con el juego de Mahjong y con el uso de Transformers en juegos de información imperfecta. A continuación, se describen las herramientas utilizadas para el desarrollo del proyecto y se explica su proceso de desarrollo, desde la preparación de los datos hasta el entrenamiento del modelo y la evaluación de los resultados. Finalmente, se presentan los resultados obtenidos y se discuten las conclusiones del proyecto.

El trabajo surgió inicialmente como un proyecto personal orientado a crear una IA de Mahjong que sirviese como apoyo para aprender a jugar mejor. A partir de esta idea inicial, el proyecto evolucionó hacia un trabajo académico de carácter exploratorio. Al comienzo no existía una arquitectura final ni un conjunto de herramientas previamente definido, por lo que las decisiones se tomaron de forma progresiva a partir de las pruebas realizadas y de los resultados obtenidos.

No se siguió una metodología formal concreta ni se estableció un calendario cerrado de hitos. El desarrollo fue iterativo e incremental: se probaron diferentes formas de representar los estados de juego, incluyendo una aproximación inicial mediante redes convolucionales, antes de adoptar una representación basada en tokens y una arquitectura Transformer. Esta solución resultó más natural para codificar los distintos elementos de un estado de Mahjong. De manera similar, la evaluación estratégica basada en *Shanten* y *Ukeire* no formaba parte del planteamiento inicial, sino que surgió durante el desarrollo al observar las limitaciones de la precisión exacta. Los cuadernos de Jupyter documentan estas iteraciones y permiten seguir la evolución del proyecto desde las primeras pruebas hasta los resultados finales.

2 Objetivos

El objetivo principal de este trabajo es estudiar la aplicación de una arquitectura Transformer a la predicción de descartes de Riichi Mahjong. Frente a aproximaciones espaciales como la empleada por Suphx [10], se utiliza una representación secuencial basada en tokens y se analizan sus posibilidades y limitaciones dentro del alcance del proyecto.

Nos planteamos los siguientes objetivos:

Cuadro 1: Objetivos del proyecto

ID	Tipo	Objetivo	Prioridad
OBJ-01	General	Desarrollar un modelo basado en Transformers capaz de predecir el descarte observado en estados de Riichi Mahjong.	Alta
OBJ-02	Funcional	Diseñar e implementar un proceso que normalice las instantáneas del dataset en una representación canónica adecuada para el modelo.	Alta
OBJ-03	Funcional	Diseñar una representación del estado del juego basada en tokens adecuada para una arquitectura Transformer.	Alta
OBJ-04	Funcional	Entrenar el modelo mediante aprendizaje por imitación utilizando partidas reales de Riichi Mahjong.	Alta
OBJ-05	Funcional	Evaluar el rendimiento del modelo mediante métricas de clasificación y métricas estratégicas basadas en Shanten y Ukeire.	Alta
OBJ-06	No funcional	Desarrollar un proceso de entrenamiento reproducible y documentado que facilite la experimentación con diferentes configuraciones del modelo.	Media
OBJ-07	General	Analizar trabajos relevantes sobre Inteligencia Artificial aplicada al Mahjong y representaciones basadas en Transformers.	Media

3 Conceptos Teóricos

En esta sección se presentan los conceptos teóricos necesarios para el proyecto. En primer lugar se resumen las reglas básicas [9] y algunas heurísticas [1] de Riichi Mahjong. A continuación se introducen el aprendizaje por imitación y la arquitectura Transformer.

3.1 Reglas Básicas

En un juego de Riichi Mahjong, contamos con 4 jugadores de los cuales 1 es el *Dealer*, representado por el viento **Este**. Cada jugador recibe 13 piezas, que pueden ser de las siguientes:

Suit	Tiles
Manzu (Caracteres)	
Pinzu (Círculos)	
Souzu (Bambús)	
Honores (Vientos y Dragones)	

Cuadro 2: Categorías de piezas de Riichi Mahjong

Las fichas de caracteres, círculos y bambús se numeran del 1 al 9. Las fichas de honores son los cuatro vientos —**Este**, **Sur**, **Oeste** y **Norte**— y los tres dragones —**Blanco**, **Verde** y **Rojo**—. Cada tipo de ficha tiene cuatro copias, por lo que el juego utiliza 136 fichas.

Una partida puede disputarse hasta la ronda Este o hasta la ronda Sur. Cada vuelta contiene normalmente cuatro manos, una por cada posición del jugador Este o *dealer*. Los jugadores ocupan los vientos Este, Sur, Oeste y Norte, como se muestra en la siguiente imagen:

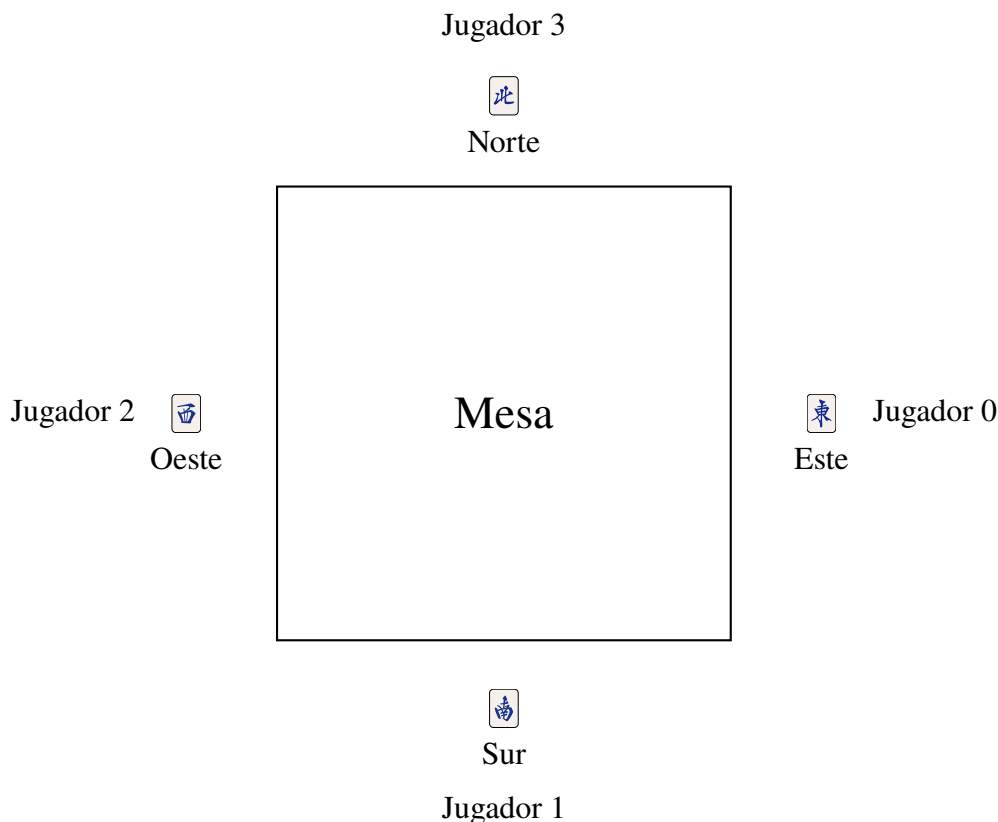


Figura 1: Posición relativa de los jugadores y vientos en una mesa de Riichi Mahjong.

Cuando el jugador Este gana una mano, conserva el puesto de *dealer* y se repite el mismo número de ronda, incrementando el contador de honba. Dependiendo del reglamento aplicado, el Este también puede conservar el puesto después de un empate si su mano se encuentra en Tenpai.

El objetivo es obtener puntos ganando manos válidas. Para poder ganar, la mano debe cumplir al menos un *yaku*, es decir, una condición de puntuación reconocida por las reglas.

La estructura más habitual está formada por cuatro combinaciones y un par, aunque existen manos especiales, como Chiitoitsu o Kokushi Musou, que siguen estructuras diferentes. Puede consultarse una relación detallada de los yaku y de su valor en la lista mantenida por Riichi Wiki[8].

Cada jugador recibe 13 fichas. En un turno normal roba una ficha y descarta otra hasta completar una mano válida o hasta que finaliza la mano.

Una mano de estructura regular está formada por cuatro *melds* o combinaciones y un par. Cada combinación puede ser una secuencia de tres fichas consecutivas del mismo palo o un trío de fichas iguales; los Kan constituyen combinaciones de cuatro fichas iguales.

Al momento de completar una mano preparada para ganar (*Tenpai*), el jugador puede ganar cogiendo la pieza de la compra normal, o del descarte de otro jugador. De este modo, los jugadores también deben evitar descartar la pieza que completa la mano de otro jugador, ya que pierde puntos de este modo. En resumen, las formas de ganar son las siguientes:

Forma de Ganar	Descripción
Tsumo	Ganar con la pieza de la compra normal.
Ron	Ganar con la pieza del descarte de otro jugador.

Cuadro 3: Formas de ganar en Riichi Mahjong

El muro incluye una sección de 14 fichas denominada pared muerta (*dead wall*). De ella se revelan los indicadores de Dora y se extraen las fichas de reemplazo asociadas a los Kan. Un indicador señala como Dora la ficha siguiente dentro de su secuencia. Para los vientos, el ciclo es Este $\rightarrow$ Sur $\rightarrow$ Oeste $\rightarrow$ Norte $\rightarrow$ Este; para los dragones, Blanco $\rightarrow$ Verde $\rightarrow$ Rojo $\rightarrow$ Blanco; y para los palos numerados, $1 \rightarrow 2 \rightarrow \dots \rightarrow 9 \rightarrow 1$.

Durante el juego pueden realizarse llamadas sobre el descarte de otro jugador. Chi permite formar una secuencia, Pon forma un trío y Daiminkan forma un Kan abierto. Estas acciones abren la mano y, por tanto, impiden declarar Riichi. También es posible declarar un Ankan con cuatro fichas propias; aunque las fichas se muestran, la mano mantiene la condición de cerrada a efectos de Riichi. Cada Kan da lugar a una ficha de reemplazo y normalmente revela un indicador de Dora adicional.

Cuando un jugador se encuentra en Tenpai con una mano cerrada puede declarar **Riichi**, depositando 1.000 puntos. Después de la declaración, normalmente debe descartar la misma ficha que roba y no puede modificar libremente la composición de la mano. Aún puede ganar mediante Tsumo o Ron y, bajo condiciones específicas que no alteren sus esperas, puede declarar determinados Ankan.

Por lo tanto, podemos ver en la siguiente tabla un resumen de las acciones posibles en cada turno:

Acción	Descripción
Comprar	Tomar una pieza de la pared.
Descartar	Descartar una pieza de la mano.
Declarar <i>Riichi</i>	Apostar 1000 puntos a que ganará la mano.
<i>Pon</i>	Coger el descarte de otro jugador para completar un trio.
<i>Chi</i>	Coger el descarte de otro jugador para completar una secuencia.
<i>Kan</i>	Completar una cuadrupla (puede ser de tu mano sin abrir).

Cuadro 4: Acciones posibles en cada turno de Riichi Mahjong

Una última regla importante es el *furiten*. Un jugador no puede ganar mediante Ron si alguna de las fichas que forman sus esperas aparece en su propio río de descartes. También puede existir Furiten temporal después de rechazar una oportunidad de Ron. Esta regla es relevante para el juego defensivo, aunque no permite determinar por completo la mano ni todas las esperas de los rivales.

Con esto finalizamos la explicación de las reglas básicas del juego, y a continuación se explicarán los conceptos de estrategia y teoría que se utilizan para jugar el juego de forma óptima.

3.2 Heurísticas

En esta subsección se definen dos conceptos importantes para medir la eficiencia de una mano: el *Ukeire* y el *Shanten*. Ambos se utilizarán posteriormente en la evaluación del modelo.

3.2.1 Ukeire

El *Ukeire* es el número de fichas aún disponibles que reducen el Shanten de una mano. Se calcula probando cada uno de los 34 tipos de ficha y descontando las copias que ya son visibles en la mano, los descartes, los melds y los indicadores de Dora. Por ejemplo, una forma incompleta  necesita un  para convertirse directamente en una secuencia; el número efectivo de copias dependerá de cuántos  sean ya visibles. En general, para un mismo valor de Shanten se prefiere un Ukeire mayor, porque ofrece más posibilidades de mejorar la mano.

3.2.2 Shanten

El *Shanten* expresa la distancia estructural de una mano respecto a Tenpai. Un valor de 0 indica que la mano ya está en Tenpai y necesita una ficha válida para ganar; un valor de 1 indica que necesita una mejora para alcanzar Tenpai, y así sucesivamente. Por tanto, reducir el Shanten acerca la estructura de la mano a una situación de victoria, aunque no representa directamente el número de turnos que serán necesarios.

3.3 Arquitectura Transformer

Los Transformers son una arquitectura de Deep Learning basada en mecanismos de *self-attention*, introducida por Vaswani et al. en el trabajo *Attention Is All You Need*[12].

A diferencia de arquitecturas recurrentes anteriores, los Transformers permiten procesar todos los elementos de una secuencia de forma simultánea, aprendiendo automáticamente las relaciones existentes entre ellos mediante el mecanismo de atención.

La arquitectura original está formada por un codificador (*encoder*) y un decodificador (*decoder*), como se muestra en la Figura 2. Sin embargo, en tareas donde únicamente es necesario comprender una secuencia de entrada, es habitual emplear únicamente el codificador, tal y como hacen modelos como BERT[2] como podemos ver en la Figura 3.

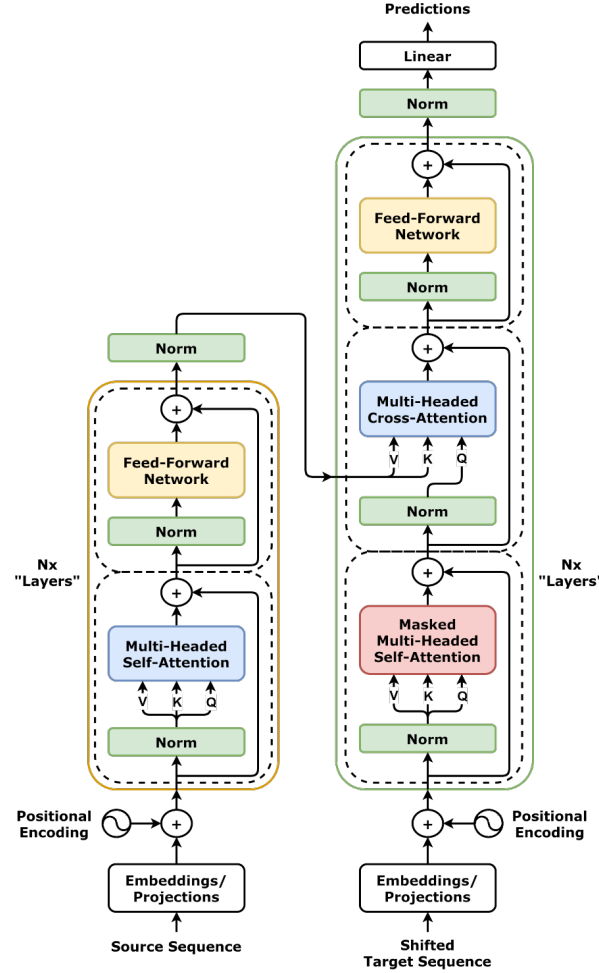


Figura 2: Arquitectura Estándar de un Transformer

Esta aproximación resulta adecuada para este trabajo, ya que el objetivo no consiste en generar texto, sino en procesar una representación del estado observable y estimar una puntuación para cada acción candidata. El modelo recibe una secuencia de tokens y aprende relaciones entre los elementos que efectivamente se incorporan a su entrada.

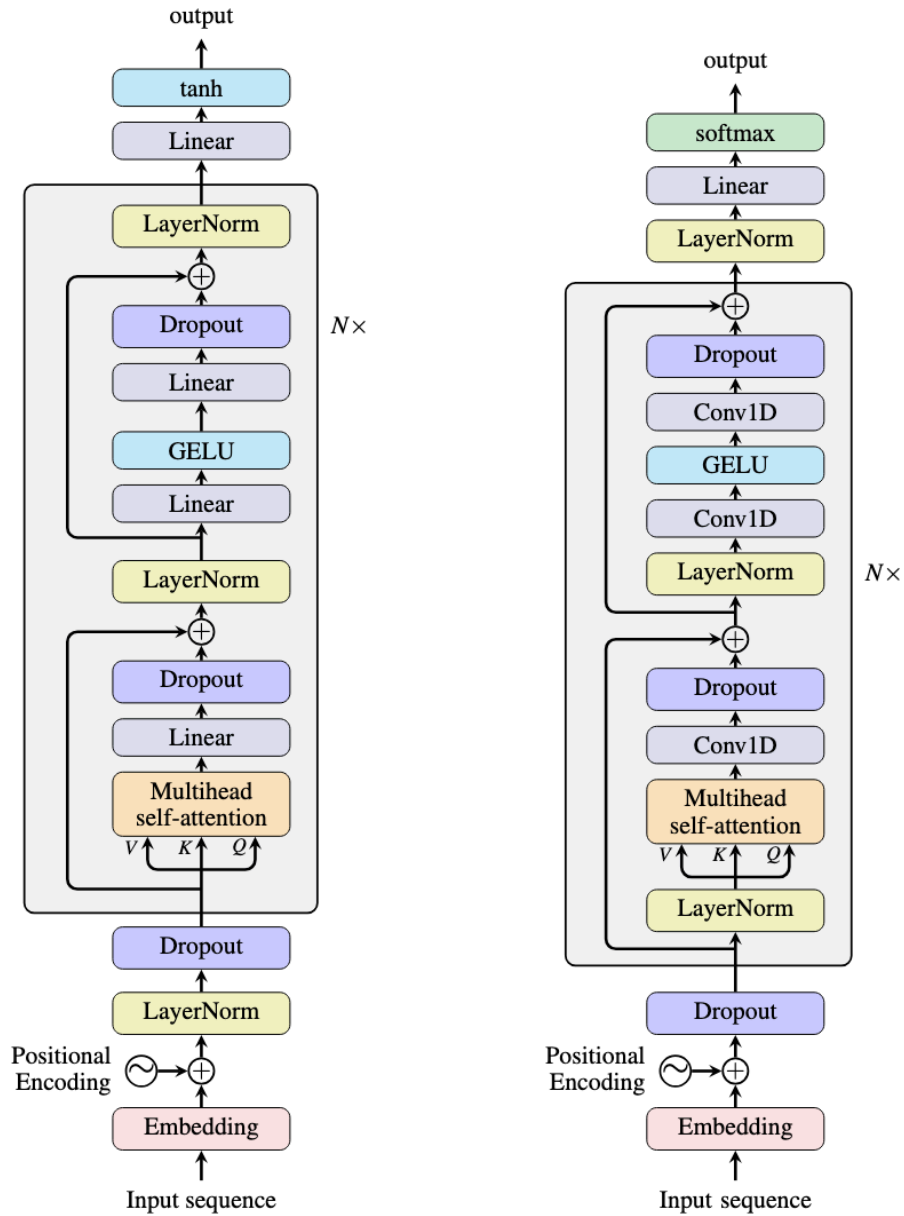


Figura 3: Arquitectura BERT (izquierda) frente a arquitectura GPT

3.3.1 Tokens

Los Transformers no procesan directamente la información original de un problema, sino una secuencia de elementos discretos denominados *tokens*. En procesamiento del lenguaje natural, un token suele corresponder a una palabra o parte de una palabra. Sin embargo, en otros dominios el concepto de token puede adaptarse para representar cualquier unidad elemental de información.

En este proyecto, un token representa un elemento específico de la instantánea disponible, como una ficha de la mano, un descarte, un meld o una acción candidata. La representación semántica intermedia contempla además información contextual, aunque no todos esos campos se incorporan a los embeddings de la implementación final.

3.3.2 Embeddings

Los modelos Transformer requieren que cada token sea representado mediante un vector numérico de dimensión fija antes de ser procesado por la red neuronal. Esta representación continua recibe el nombre de *embedding*.

Durante el entrenamiento, el modelo aprende automáticamente un embedding para cada tipo de token, de forma que aquellos con significado o función similar tienden a situarse próximos entre sí en el espacio vectorial. Esta representación permite capturar relaciones semánticas entre los distintos elementos del estado del juego.

3.3.3 Aprendizaje por Imitación

El aprendizaje por imitación (*Imitation Learning*) consiste en entrenar un modelo para reproducir las decisiones tomadas por un experto a partir de ejemplos previamente observados[7]. En lugar de descubrir una estrategia mediante interacción con el entorno, como ocurre en el aprendizaje por refuerzo, el modelo aprende directamente una función que aproxima el comportamiento del experto.

En el ámbito de los juegos de estrategia, este enfoque permite entrenar modelos utilizando grandes conjuntos de partidas reales, facilitando el aprendizaje de reglas, patrones tácticos y principios estratégicos implícitos en las decisiones de jugadores experimentados.

En este proyecto se emplea aprendizaje por imitación utilizando estados de juego obtenidos de partidas registradas en Tenhou.net. El objetivo del modelo consiste en aproximar el descarte observado en el dataset para cada instantánea disponible.

4 Estado del Arte

El desarrollo de sistemas de Inteligencia Artificial capaces de jugar al Riichi Mahjong ha experimentado un avance significativo durante los últimos años gracias al empleo de técnicas de aprendizaje profundo y aprendizaje por refuerzo. Entre los modelos más representativos destacan Suphx, Mortal y NAGA, que representan diferentes aproximaciones al problema.

4.1 Suphx

Suphx[10], desarrollado por Microsoft Research, constituye uno de los primeros sistemas capaces de alcanzar un nivel profesional en Riichi Mahjong. Su arquitectura combina redes neuronales convolucionales (CNN) con técnicas de aprendizaje por refuerzo, representando el estado de la partida mediante una codificación espacial similar a una imagen.

Gracias a esta aproximación, Suphx consiguió situarse entre el 0.1% de mejores jugadores de la plataforma Tenhou.net, convirtiéndose en un referente dentro de la investigación en Inteligencia Artificial aplicada al Mahjong.

Su representación espacial requiere transformar el estado del juego en tensores multidimensionales adecuados para una red convolucional. Esta característica motivó que, durante el presente proyecto, se explorasen alternativas basadas en secuencias de tokens.

4.2 Transformers aplicados al Mahjong

La aparición de la arquitectura Transformer permitió explorar nuevas formas de representar el estado de una partida sin necesidad de convertirlo en una estructura espacial.

Un trabajo relevante en esta dirección es Tjong[11], que utiliza un Transformer y un esquema jerárquico para separar la selección del tipo de acción y la selección de fichas. Tjong fue diseñado para las reglas del Mahjong estándar chino, que difieren del Riichi Mahjong en aspectos importantes como la puntuación, la Dora o el Furiten. Por ello sirve como referencia arquitectónica, pero no como solución directamente transferible al problema estudiado.

Otro modelo destacado es Mortal[5], un proyecto de código abierto para Riichi Mahjong basado en aprendizaje por refuerzo profundo. Su implementación utiliza una codificación específica del estado y componentes convolucionales de tipo ResNet, por lo que no constituye un antecedente directo de la representación con tokens desarrollada en este trabajo. No obstante, es una referencia relevante por ofrecer una implementación abierta de un agente completo y por documentar aspectos de simulación, entrenamiento e integración con el entorno[4].

Por otra parte, NAGA[3] es un motor comercial ampliamente utilizado por jugadores de Riichi Mahjong para analizar partidas y estudiar estrategias. Aunque ha demostrado un elevado nivel de juego, su arquitectura y proceso de entrenamiento no son públicos, por lo que únicamente puede considerarse como una referencia del estado actual de la tecnología y no como un modelo reproducible desde el punto de vista científico.

4.3 Posicionamiento del presente trabajo

A diferencia de Suphx, el presente trabajo no emplea representaciones espaciales mediante CNN, sino una representación basada en tokens procesada directamente por un Transformer de tipo encoder.

Asimismo, frente a modelos como Mortal o NAGA, cuyo objetivo principal consiste en maximizar el rendimiento competitivo mediante arquitecturas complejas y técnicas avanzadas de entrenamiento, este proyecto persigue un objetivo diferente: desarrollar un modelo reproducible y comprensible desde un punto de vista académico que permita estudiar la aplicación de Transformers al problema del descarte en Riichi Mahjong.

Para ello se emplea aprendizaje por imitación sobre partidas reales de Tenhou.net, utilizando una representación explícita del estado del juego y evaluando el modelo no únicamente mediante precisión de clasificación, sino también mediante métricas estratégicas propias del dominio, como Shanten y Ukeire.

5 Herramientas y Entorno de Desarrollo

El desarrollo del proyecto se ha realizado principalmente utilizando el lenguaje de programación Python, empleando un entorno virtual gestionado mediante Conda y aceleración por GPU mediante CUDA para el entrenamiento de los modelos desarrollados con la biblioteca PyTorch.

El proceso de trabajo ha sido eminentemente iterativo y experimental, sin seguir una metodología formal concreta. En lugar de partir de un diseño definitivo, el proyecto avanzó mediante pruebas sucesivas documentadas en cuadernos de Jupyter Notebook. Estos cuadernos permiten registrar las principales etapas del desarrollo: exploración inicial del conjunto de datos, pruebas de diferentes representaciones del estado, construcción progresiva de la arquitectura Transformer, validación mediante pruebas de sobreajuste, entrenamiento a diferentes escalas y análisis estratégico de los resultados obtenidos.

Este enfoque ha facilitado la experimentación continua con distintos tamaños de conjunto de datos, configuraciones de entrenamiento y métricas de evaluación. Algunas decisiones

importantes, como el uso de una representación mediante tokens o la incorporación de la evaluación estratégica, surgieron como resultado de estas iteraciones y no de una planificación cerrada desde el inicio.

Todo el código fuente del proyecto, así como los distintos experimentos realizados, se encuentran versionados mediante Git y alojados en un repositorio público de GitHub:

<https://github.com/Ruipi/tfg>

Los experimentos de entrenamiento se ejecutaron sobre un equipo equipado con una GPU NVIDIA GeForce RTX 5060 Laptop, lo que permitió acelerar significativamente el proceso de entrenamiento del modelo Transformer y realizar múltiples iteraciones experimentales en tiempos razonables.

Elemento	Herramienta
Lenguaje	Python 3.11
Gestión de entorno	Conda (Miniforge)
Deep Learning	PyTorch
GPU	CUDA
Visualización	Matplotlib, Seaborn
Análisis de datos	Pandas, NumPy
Base de datos	SQLite
Desarrollo experimental	Jupyter Notebook
Control de versiones	Git + GitHub
Editor	Visual Studio Code

Cuadro 5: Tecnologías empleadas durante el desarrollo del proyecto.

6 Desarrollo del Proyecto

En esta sección explicaremos como ha sido desarrollado el proyecto, empezando por el Análisis de Exploración de Datos (EDA); el planteamiento de la arquitectura, donde se verán las decisiones tomadas para la construcción del modelo; la construcción del modelo; y las iteraciones de desarrollo.

6.1 Análisis de Exploración de Datos

El conjunto de datos utilizado procede de registros de partidas de Tenhou recopilados mediante la herramienta *phoenix-logs* y transformados a una base de datos SQLite mediante *tenhou_dataprocess*. El dataset se distribuye a través de Kaggle con licencia Apache 2.0 y contiene ocho tablas asociadas a diferentes tipos de decisiones: *Discard*, *Riichi*, *Chi*, *Pon*, *DaiMinKan*, *ShouMinKan*, *AnKan* y *Skip*. Cada registro almacena un estado de juego serializado en formato JSON y comprimido mediante gzip.

En este proyecto se utiliza exclusivamente la tabla *Discard*. A partir de ella se construyó una base de datos filtrada que conserva únicamente los estados con más de una alternativa de descarte, obteniéndose un total de 552.563 estados.

Cuadro 6: Schemas del Dataset

ID	Nombre	Descripción
0	Skip	No realizar ninguna acción; pasar el turno.
1	Discard	Descartar una ficha de la mano.
2	Chi	Robar la ficha descartada por el jugador anterior para formar una secuencia.
3	Pon	Robar la ficha descartada por cualquier jugador para formar un trío.
4	DaiMinKan	Robar la ficha descartada por cualquier jugador para formar un cuádruple abierto.
5	ShouMinKan	Añadir una ficha propia a un trío abierto (Pon) para formar un cuádruple.
6	AnKan	Declarar un cuádruple cerrado con cuatro fichas de la propia mano.
7	Riichi	Declarar <i>Tenpai</i> con la mano cerrada, apostando 1000 puntos.

Y a partir de ellos, se analizaron los siguientes datos.

6.2 Distribución de Acciones

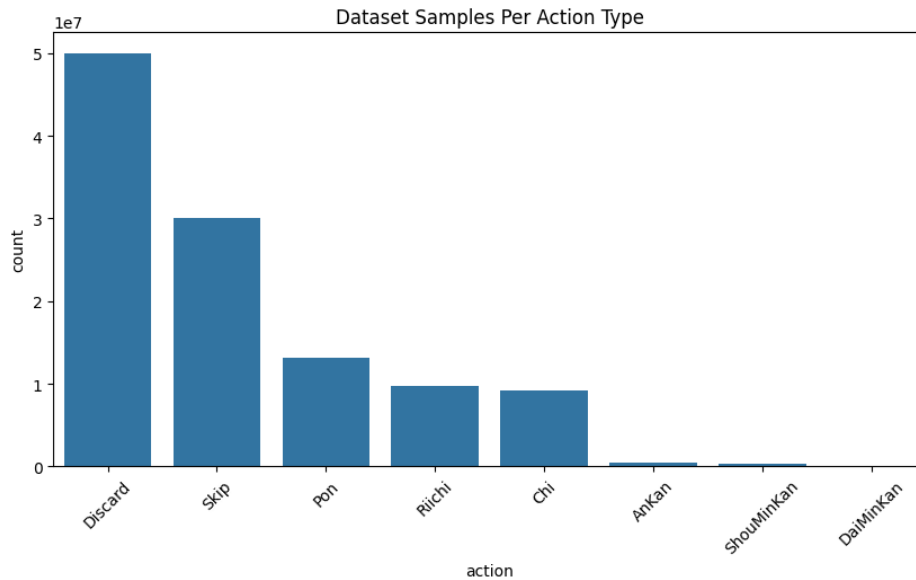


Figura 4: Distribución de Número de Datos del Dataset por tipo de Acciones Válidas

En este gráfico se observa un fuerte desequilibrio entre los tipos de acciones. Los descartes son mucho más frecuentes que las llamadas o los distintos tipos de Kan, por lo que el presente trabajo se centra exclusivamente en los estados cuya acción observada es un descarte. El tratamiento conjunto de todas las decisiones requeriría considerar este desequilibrio y queda fuera del alcance del modelo desarrollado.

6.3 Distribución de la Suma de Acciones Válidas de una Sample

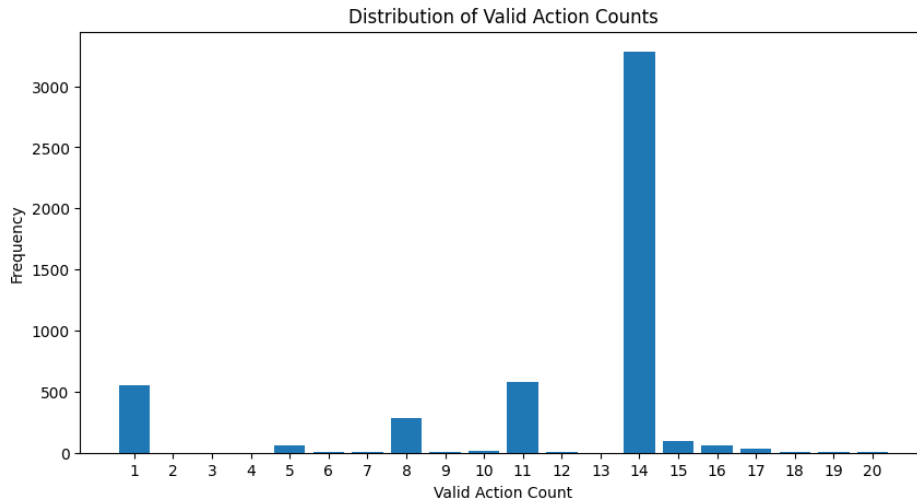


Figura 5: Distribución del número de acciones válidas del schema de Descarte

La mayor concentración ocurre en 14 acciones válidas, ya que una mano cerrada dispone normalmente de 14 fichas antes del descarte. También aparecen concentraciones en 11, 8 y 5 alternativas, asociadas a manos con una, dos o tres combinaciones abiertas, respectivamente, como se resume en la siguiente tabla:

Cuadro 7: Acciones válidas según melds abiertos

Acciones válidas	Melds abiertos
11	1
8	2
5	3
2	4

Los estados con dos alternativas son menos frecuentes y suelen corresponder a manos con cuatro combinaciones ya formadas.

La base original contiene además numerosos estados con una única alternativa de descarte, situación habitual después de declarar *Riichi*. Como en esos casos no existe una elección real entre varios descartes, se eliminaron para evitar que incrementasen artificialmente la precisión. La base filtrada conserva únicamente estados con más de una acción de tipo descarte.

Finalmente, el número de acciones válidas restante se explica debido a posibles acciones de *Riichi* con diferentes piezas.

6.4 Frecuencia de Piezas Descartadas

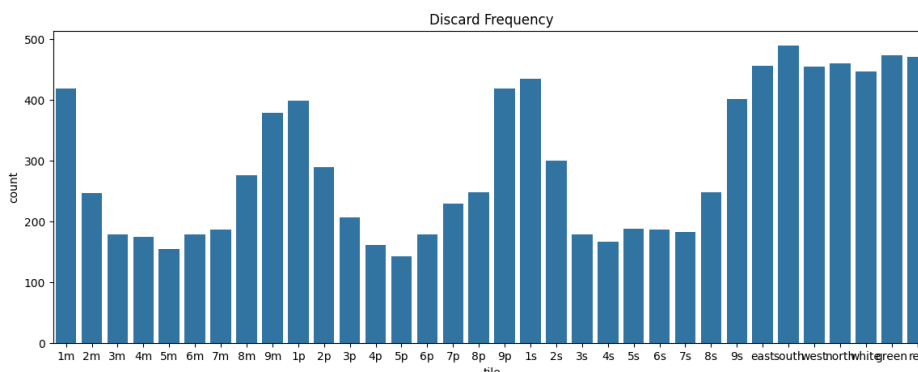


Figura 6: Frecuencia de las Piezas Descartadas

En este gráfico se observa que las fichas descartadas con mayor frecuencia son los honores (vientos y dragones), véase la Tabla 2, y las fichas terminales (unos y nueves). Una posible explicación es que, en muchas configuraciones de mano, estas fichas ofrecen menos posibilidades de formar secuencias que las fichas numeradas intermedias. Esta interpretación es descriptiva y no implica que deban descartarse siempre, ya que la decisión depende del estado concreto de la partida.

6.5 Representación de un Estado

Esta es la representación de un estado de mahjong dentro del Dataset:

Listing 1: Ejemplo de un estado

```
1 {
2   { 'round_wind': 0,
3   'num_honba': 1,
4   'num_riichi': 0,
5   'player_wind': 0,
6   'position': 0,
7   'remain_tiles': 15,
8   'dora_indicators': [7, 5],
9   'hand_tiles': [1, 1, 2, 3, 3, 12, 13, 18, 19, 22, 22],
10  'players': { '0': { 'points': 41700,
11    'riichi': False,
12    'discards': [28, 27, 9, 16, 10, 6, 15, 20, 13, 32, 32, 0, 33, 33],
13    'melds': [{ 'type': 2, 'tiles': [92, 98, 103, -1], 'who': [0, 0, 3, -1] }],
14    'tsumo_giri': [0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0]},
15  '1': { 'points': 19000,
16    'riichi': False,
17    'discards': [30, 27, 32, 18, 31, 17, 26, 26, 24, 2, 0, 14, 30, 10],
18    'melds': [],
19    'tsumo_giri': [0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0]},
20  '2': { 'points': 20300,
```

```

21     'riichi': False,
22     'discards': [18, 17, 11, 16, 24, 25, 28, 23, 33, 32, 20, 7, 30,
23                  17],
24     'melds': [{ 'type': 5, 'tiles': [124, 126, 127, 125], 'who': [2,
25                  2, 1, 2]}],
26     { 'type': 2, 'tiles': [5, 13, 10, -1], 'who': [2, 2, 1, -1]}],
27     'tsumo_giri': [0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1]},
28     '3': { 'points': 19000,
29            'riichi': False,
30            'discards': [28, 29, 30, 18, 0, 11, 9, 27, 27, 0, 29, 26, 24, 25
31                        ],
32            'melds': [],
33            'tsumo_giri': [0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1]}},
34     'valid_actions': [{ 'type': 1, 'tiles': [1], 'who': [0, -1, -1, -1]
35                        },
36                        { 'type': 1, 'tiles': [1], 'who': [0, -1, -1, -1]},
37                        { 'type': 1, 'tiles': [2], 'who': [0, -1, -1, -1]},
38                        { 'type': 1, 'tiles': [3], 'who': [0, -1, -1, -1]},
39                        { 'type': 1, 'tiles': [3], 'who': [0, -1, -1, -1]},
40                        { 'type': 1, 'tiles': [12], 'who': [0, -1, -1, -1]},
41                        { 'type': 1, 'tiles': [13], 'who': [0, -1, -1, -1]},
42                        { 'type': 1, 'tiles': [18], 'who': [0, -1, -1, -1]},
43                        { 'type': 1, 'tiles': [19], 'who': [0, -1, -1, -1]}],
44     'action_idx': 7}
45 }

```

La instantánea no se entrega directamente al Transformer. Primero se descomprime y normaliza, después se convierte en tokens y finalmente se tensorizan los seis identificadores utilizados por los embeddings. Este recorrido se representa en la Figura 7; los campos que permanecen únicamente en la representación intermedia se detallan más adelante en la sección de tensorización.

De este registro se obtiene la siguiente información:

Cuadro 8: Campos de un Estado de Mahjong

Campo	Tipo	Descripción
round_wind	int	Identificador de ronda utilizado por el dataset
num_honba	int	Número de honba
num_riichi	int	Número de riichi's (0-3)
player_wind	int	Viento del jugador (0-3)
position	int	Posición del jugador (0-3)
remain_tiles	int	Número de fichas restantes en el muro (0-70)
dora_indicators	list[int]	Indicadores de dora en codificación Tenhou (0-135)
hand_tiles	list[int]	Fichas de la mano en codificación Tenhou (0-135)
players	dict[int, dict]	Información de los jugadores, incluyendo puntos, riichi, descartes, melds y tsumo_giri.
valid_actions	list[dict]	Lista de acciones válidas para el jugador actual.
action_idx	int	Índice de la acción tomada por el jugador actual.

De estos datos, podemos aún extraer los campos de players:

Cuadro 9: Campos de un Jugador en un Estado de Mahjong

Campo	Tipo	Descripción
points	int	Puntos del jugador (0-50000)
riichi	bool	Indica si el jugador ha declarado riichi (True/False)
discards	list[int]	Lista de fichas descartadas por el jugador (0-33)
melds	list[dict]	Lista de combinaciones del jugador, cada una con tipo, fichas y procedencia.
tsumo_giri	list[int]	Lista indicando si cada ficha fue descartada inmediatamente después de ser robada (1) o no (0).

Campos de melds:

Cuadro 10: Campos de un Meld en un Estado de Mahjong

Campo	Tipo	Descripción
type	int	Tipo de meld (2: Pon, 3: Chii, 4: DaiMinKan, 5: ShouMinKan, 6: AnKan)
tiles	list[int]	Fichas que componen el meld en codificación Tenhou (0-135)
who	list[int]	Lista indicando quién contribuyó a cada ficha del meld (-1 para fichas propias, 0-3 para otros jugadores)

Y, por fin, campos de valid_actions:

Cuadro 11: Campos de una Acción Válida en un Estado de Mahjong

Campo	Tipo	Descripción
type	int	Tipo de acción (0: Skip, 1: Discard, 2: Chi, 3: Pon, 4: DaiMinKan, 5: ShouMinKan, 6: AnKan, 7: Riichi, 8: Ron, 9: Tsumo, 10: Kyushukyuhai, 11: Chankan)
tiles	list[int]	Fichas involucradas en la acción en codificación Tenhou (0-135)
who	list[int]	Lista indicando quién realiza la acción y quién contribuye a ella (-1 para el jugador actual, 0-3 para otros jugadores)

7 Arquitectura del Modelo

Como se explicó en la Subsección 3.3, el modelo desarrollado en este trabajo utiliza una arquitectura basada en el codificador de un Transformer. La elección de una arquitectura de tipo *encoder-only* se debe a que el objetivo del sistema no consiste en generar una secuencia de salida, sino en analizar la representación disponible de un estado de Riichi Mahjong y puntuar las acciones candidatas proporcionadas por el dataset.

La Figura 7 muestra el flujo general de procesamiento, desde la normalización de la instantánea almacenada en el conjunto de datos hasta la predicción final realizada por el modelo.

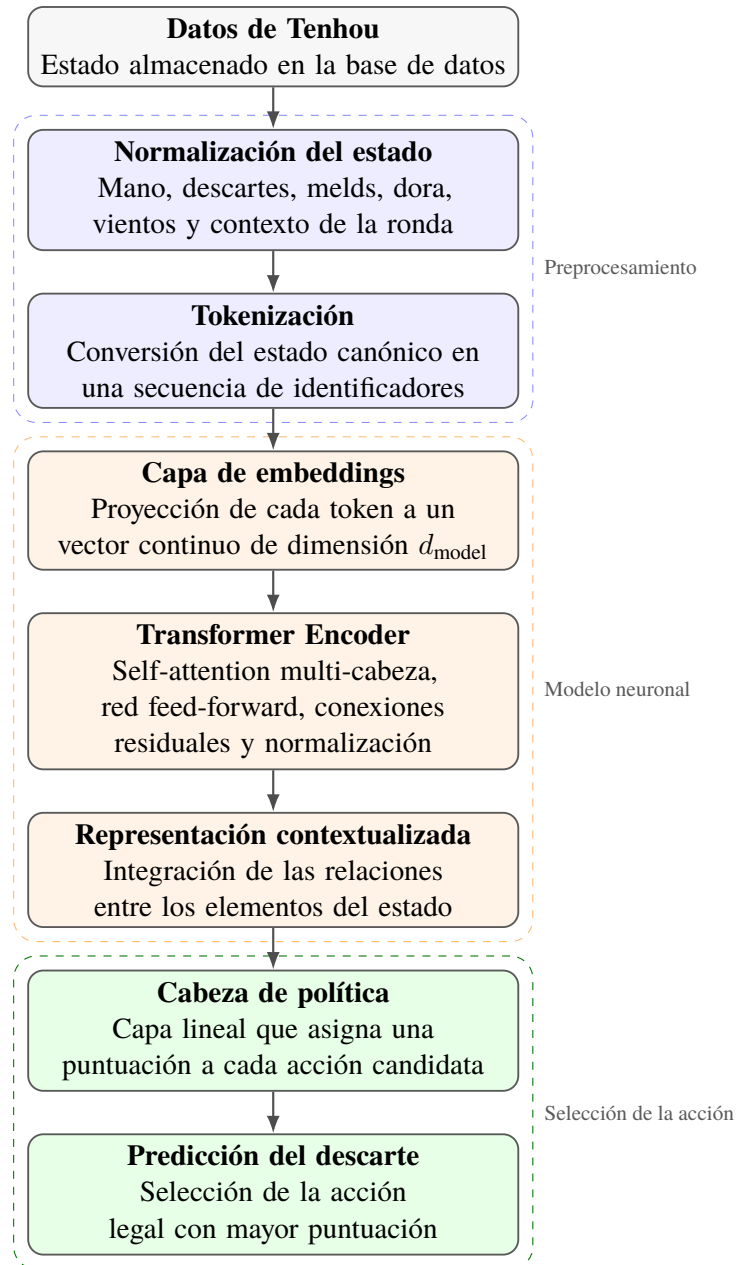


Figura 7: Flujo de procesamiento y arquitectura del modelo propuesto.

7.1 Reconstrucción del estado de juego

Los registros del conjunto de datos no pueden utilizarse directamente como entrada de una red neuronal, ya que contienen la información de la partida en un formato estructurado que debe ser interpretado y normalizado previamente.

Por este motivo, se desarrolló un proceso de normalización encargado de transformar cada instantánea del dataset en una representación canónica. Esta representación intermedia reúne la mano del jugador, los descartes visibles, las combinaciones abiertas, los indicadores de Dora, los vientos y otros elementos contextuales. No reconstruye la partida desde un registro de eventos ni recupera información oculta.

7.2 Tokenización del estado

Una vez normalizada la instantánea, su información se transforma en una secuencia de tokens mediante un tokenizador desarrollado específicamente para este proyecto.

Cada token representa una unidad semántica del estado de juego. Se crean tokens para la mano, los descartes, las combinaciones, las acciones candidatas y ciertos elementos contextuales. Sin embargo, como se detalla más adelante, la implementación final solo tensoriza una parte de los metadatos almacenados en estos objetos intermedios.

Esta representación simbólica se eligió como alternativa a las codificaciones espaciales empleadas por modelos basados en redes convolucionales. En lugar de transformar el estado del Mahjong en una estructura bidimensional similar a una imagen, la tokenización permite representar sus componentes de forma explícita y compacta.

La secuencia resultante mantiene diferenciada la naturaleza de cada elemento del estado y facilita la incorporación de nueva información sin necesidad de rediseñar completamente la estructura de entrada.

7.3 Capa de embeddings

Los identificadores discretos obtenidos durante la tokenización no pueden ser procesados directamente por el Transformer. Por ello, cada token se proyecta a un vector continuo de dimensión fija mediante una capa de embeddings entrenable.

Durante el entrenamiento, el modelo ajusta estas representaciones para capturar relaciones entre los distintos elementos del estado. De esta forma, tokens asociados a fichas, zonas del tablero o situaciones estratégicas relacionadas pueden adquirir representaciones útiles para la predicción.

La salida de esta etapa es una matriz de vectores, donde cada fila corresponde a la representación numérica de uno de los tokens del estado.

7.4 Transformer Encoder

La secuencia de embeddings se introduce posteriormente en un Transformer Encoder. Este bloque está compuesto por varias capas que combinan mecanismos de *multi-head self-attention*, redes neuronales *feed-forward*, conexiones residuales y normalización.

El mecanismo de autoatención permite que cada elemento de la entrada tenga en cuenta la información proporcionada por el resto de tokens. En la implementación final, el modelo relaciona principalmente las fichas de la mano, los descartes visibles, el tipo y propietario de las combinaciones y las acciones candidatas. Otros campos contextuales permanecen en la representación intermedia, pero no se incorporan aún al tensor de entrada.

Por tanto, la salida del codificador no representa cada token de forma aislada, sino contextualizada respecto a la información que efectivamente recibe el modelo.

7.5 Cabeza de política y acciones legales

La representación contextual producida por el Transformer se utiliza para asignar una puntuación a las acciones de descarte disponibles. Esta parte final del modelo recibe el nombre de cabeza de política o *policy head*.

Para cada muestra se utiliza el conjunto de acciones candidatas proporcionado por el dataset. Aunque la acción objetivo de todos los registros de la tabla *Discard* es un descarte, algunos estados incluyen además candidatos de Riichi o Kan. La cabeza de política calcula

un valor o *logit* para cada candidato. Como su número puede variar entre estados, la salida del modelo también presenta una longitud variable.

La acción predicha corresponde al descarte con mayor puntuación:

$$\hat{a} = \arg \max_{a \in A(s)} z_a, \quad (1)$$

donde $A(s)$ representa el conjunto de acciones candidatas para el estado s y z_a el logit asignado por la cabeza de política a la acción a .

Durante el entrenamiento, los logits generados por el modelo se comparan con la acción observada en el dataset mediante una función de pérdida de entropía cruzada (*Cross-Entropy Loss*) [6]. Esta función aplica internamente una normalización Softmax sobre los logits para obtener una distribución de probabilidad y penaliza aquellas predicciones que asignan una baja probabilidad a la acción objetivo. Como consecuencia, el modelo aprende progresivamente a incrementar la puntuación de las decisiones observadas en el conjunto de entrenamiento.

7.6 Justificación de las decisiones arquitectónicas

La arquitectura fue diseñada atendiendo tanto a las características del problema como a los recursos disponibles para el desarrollo del proyecto.

En primer lugar, se eligió un Transformer Encoder porque el problema se formula como una tarea de clasificación condicionada por una representación del estado observable. No es necesario generar una secuencia ni predecir estados futuros, por lo que una arquitectura *encoder-decoder* introduciría complejidad adicional sin aportar una ventaja directa para este objetivo.

En segundo lugar, la representación mediante tokens permite mantener una correspondencia clara entre los elementos del estado y la información procesada por el modelo. Esta decisión mejora la interpretabilidad del preprocesamiento y evita la necesidad de construir una representación espacial específica para redes convolucionales.

Finalmente, la predicción se limita a las acciones legales de cada estado. Esta elección evita que el modelo tenga que asignar probabilidad a descartes imposibles y adapta la salida a la naturaleza variable de las decisiones disponibles en una partida de Mahjong.

En conjunto, la arquitectura propuesta busca alcanzar un equilibrio entre capacidad de representación, coste computacional y claridad de implementación.

7.7 Diseño conceptual de la tokenización

Durante el diseño se definieron las siguientes categorías de información. Esta sección documenta la representación semántica intermedia y las alternativas consideradas; no todos sus campos fueron incorporados finalmente a los embeddings utilizados por los checkpoints.

Categoría	Ejemplos	Ordenado	Público	Dinámico
Piezas de mano	5m, este	No	No	Sí
Descartes	descarte 9m	Sí	Sí	Sí
Melds	pon/chi/kan	Parcialmente	Sí	Sí
Acciones candidatas	descartar 5m	No	N/A	Sí
Contexto	Viento de la Ronda	No	Sí	Lento
Estatus de los jugadores	riichi, puntos	No	Sí	Sí

Cuadro 12: Categorías de Informaciones

Un estado de Mahjong se representa como una colección de tokens semánticos. Cada token representa una categoría diferente de información de juego y puede contener metadatos asociados.

A continuación veremos, para cada categoría, cuáles son los campos que tendrá el token.

7.7.1 Tokens de Mano

Las piezas de la mano representan la identidad de la pieza, una distinción entre duplicados (qué copia de la pieza es) y un indicador de si se trata de una Dora roja.

Campo	Objetivo
token_type	identifica el token HAND
tile_type	pieza de Mahjong
copy_index	diferencia copias de piezas
is_red	soporte de dora rojo

Cuadro 13: Campos de Token de Mano

7.7.2 Tokens de Descarte

Los tokens de descarte representan eventos públicos y ordenados. Los campos de un token de descarte son los siguientes:

Campo	Objetivo
token_type	identifica el token DISCARD
player	jugador que ha descartado la pieza
tile_type	pieza de Mahjong
copy_index	diferencia copias de piezas
is_red	soporte de dora rojo
is_tsumogiri	si la pieza descartada fue la ficha recién robada
is_riichi_declaration	si al descartar esa pieza se declara riichi
global_order	orden cronológico del descarte en la ronda
player_discard_order	índice de descarte para el jugador

Cuadro 14: Campos de Token de Descarte

Campo	Objetivo
token_type	identifica el token MELD
player	jugador que posee el meld
meld_type	chi / pon / kan
tile_types	piezas que componen el meld
copy_indices	distinción entre duplicados
red_flags	soporte de dora rojo
source_players	jugadores que contribuyeron para cada pieza
global_order	orden cronológico de la llamada

Cuadro 15: Campos de Token de Melds

7.7.3 Tokens de Melds

Los Tokens de Melds representan eventos de melds que son públicos y ordenados, los campos de un token de meld son los siguientes:

7.7.4 Tokens de Acciones

Los Tokens de Acciones representan eventos de acciones candidatas que son públicos y ordenados, los campos de un token de acción son los siguientes:

Campo	Objetivo
token_type	identifica el token ACTION
acting_player	jugador que realiza la acción
action_index	índice dentro del conjunto de candidatos legales
action_type	descartar / riichi / pon / etc
tile_types	piezas involucradas en la acción
copy_indices	distinción de duplicados
red_flags	soporte de dora rojo
source_players	fuelle de las piezas llamadas
global_order	orden cronológico de la acción

Cuadro 16: Campos de Token de Acciones

7.7.5 Tokens de Estado de Juego

Los Tokens de Estado de Juego representan información contextual del juego que es pública y no ordenada, los campos de un token de estado de juego son los siguientes:

Campo	Objetivo
token_type	identifica el token GAME_STATE
round_wind	ronda Este/Sur
honba_count	honba actual
riichi_sticks	depósitos de riichi
remaining_tiles	piezas restantes en la pared
dora_indicators	indicadores de dora actuales

Cuadro 17: Campos de Token de Estado de Juego

7.7.6 Tokens de Estado de Jugadores

Los Tokens de Estado de Jugadores representan información contextual de cada jugador que es pública y no ordenada, los campos de un token de estado de jugadores son los siguientes:

Campo	Objetivo
token_type	identifica el token PLAYER_STATE
player	identidad del jugador
seat_wind	este/sur/oeste/norte
score	puntos actuales
riichi_status	si el jugador ha declarado riichi
placement	posición actual
open_hand	si el jugador tiene mano abierta

Cuadro 18: Campos de Token de Estado de Jugadores

7.8 Diseño conceptual del embedding

Inicialmente se planteó una composición de embeddings semánticos y de metadatos con la siguiente estructura general:

$$\text{Embedding}(\text{token}) = \text{token_type_embedding} + \text{semantic_embedding} + \text{metadata_embedding} + \text{positional_embedding}$$

Los distintos tipos de tokens utilizarían diferentes embeddings en función de su estructura semántica.

Cada categoría de token tendrá un embedding dedicado de tipo de token, que podrá ser uno de los siguientes: HAND, DISCARD, MELD, ACTION, GAME_STATE o PLAYER_STATE.

A continuación se documentan las combinaciones consideradas para cada token semántico. Estas expresiones pertenecen al diseño conceptual; la Subsección siguiente describe la implementación que se utilizó realmente para entrenar y evaluar los modelos.

7.8.1 Embedding de Tokens de Piezas

Este embedding codifica la identidad de la pieza, por ejemplo: 1m, 5p, east, red_dragon. Los embeddings de piezas son compartidos entre los tokens de mano, descarte, melds y de acciones.

7.8.2 Embedding del índice de copia

Instancias de piezas duplicadas (por ejemplo, dos 5m) se distinguen mediante un índice de copia. Este embedding permite al modelo diferenciar entre estas instancias.

7.8.3 Embedding de Jugador

Este embedding codifica la identidad del jugador, por ejemplo, jugador 0, jugador 1, etc. Los embeddings de jugadores son compartidos entre los tokens de descarte, melds, acciones y estado de jugadores.

7.8.4 Embeddings Posicionales

Los embeddings posicionales preservan el orden temporal de eventos como los descartes, el momento en que se realizaron los melds y la secuencia de acciones. Se plantearon dos esquemas posicionales: el orden cronológico global y el orden local de descartes de cada jugador.

7.8.5 Embeddings de Metadatos

Los embeddings de metadatos codifican información adicional relevante para cada token, como indicadores de dora, estado de riichi, y otros atributos contextuales. Estos embeddings permiten al modelo capturar relaciones complejas entre los tokens y el contexto del juego.

7.8.6 Embedding de Token de Mano

Los tokens de mano representan piezas escondidas del jugador actual y es definida por la siguiente estructura de embedding:

$$E_{\text{hand}} = E_{\text{token_type}} + E_{\text{tile}} + E_{\text{copy}} + E_{\text{red}}$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo HAND,
- E_{tile} representa la identidad semántica de la pieza,
- E_{copy} distingue entre instancias duplicadas de piezas,
- E_{red} indica si la pieza es un dora rojo.

7.8.7 Embedding de Token de Descarte

Los tokens de descarte representan eventos de descartes que son públicos y ordenados, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_{\text{discard}} = & E_{\text{token_type}} + E_{\text{tile}} + E_{\text{copy}} + \\ & E_{\text{red}} + E_{\text{player}} + E_{\text{tsumogiri}} + \\ & E_{\text{riichi_declaration}} + E_{\text{global_position}} + E_{\text{local_position}} \end{aligned} \quad (2)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo DISCARD,
- E_{tile} representa la identidad semántica de la pieza,
- E_{copy} distingue entre instancias duplicadas de piezas,
- E_{red} indica si la pieza es un dora rojo,
- E_{player} identifica al jugador que ha descartado la pieza,
- $E_{\text{tsumogiri}}$ indica si la ficha descartada fue la ficha recién robada,

- E_riichi_declaration indica si al descartar esa pieza se declara riichi,
- E_global_position representa el orden cronológico global del descarte,
- E_local_position representa el orden de descarte relativo al jugador.

7.8.8 Embedding de Token de Meld

Tokens de Melds representan llamadas hechas durante la partida, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_meld = E_token_type + E_player + E_meld_type + \\ E_tile_set + E_copy_set + E_red_set + \\ E_source_players + E_global_position \end{aligned} \quad (3)$$

Dónde:

- E_token_type identifica el token como una entidad de tipo MELD,
- E_player identifica al jugador que posee el meld,
- E_meld_type representa el tipo de meld (chi, pon, kan, etc),
- E_tile_set codifica las piezas que componen el meld,
- E_copy_set distingue entre instancias duplicadas de piezas,
- E_red_set codifica la información de dora rojo,
- E_source_players identifica los jugadores que contribuyeron a cada pieza del meld,
- E_global_position representa el orden cronológico de la llamada.

7.8.9 Embedding de Token de Acción

Tokens de Acciones representan eventos de acciones candidatas que son públicos y ordenados, y su embedding es definido por la siguiente estructura:

$$\begin{aligned} E_action = E_token_type + E_action_type + \\ E_tile_set + E_copy_set + E_red_set + \\ E_source_players + E_global_position \end{aligned} \quad (4)$$

Dónde:

- E_token_type identifica el token como una entidad de tipo ACTION,
- E_action_type representa descarte, riichi, chi, pon, kan, etc,
- E_tile_set representa las piezas involucradas en la acción,
- E_copy_set distingue entre instancias duplicadas de piezas,
- E_red_set codifica la información de dora rojo,
- E_source_players identifica las fuentes de las piezas para llamadas,
- E_global_position representa el timing de la acción.

7.8.10 Embedding de Token de Estado de Juego

Tokens de Estado de Juego representan información contextual del juego que es pública y no ordenada, y su embedding es definido por la siguiente estructura:

$$E_{\text{game}} = E_{\text{token_type}} + E_{\text{round}} + E_{\text{honba}} + E_{\text{riichi_sticks}} + E_{\text{remaining_tiles}} + E_{\text{dora_set}} \quad (5)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo `GAME_STATE`,
- E_{round} codifica el viento de la ronda (este/sur),
- E_{honba} representa el número de honba actuales,
- $E_{\text{riichi_sticks}}$ codifica la cantidad de depósitos de riichi,
- $E_{\text{remaining_tiles}}$ representa las piezas restantes en la pared,
- $E_{\text{dora_set}}$ codifica los indicadores de dora actuales.

7.8.11 Embedding de Token de Estado de Jugador

El Token de Estado de Jugador representa información contextual persistente asociada con cada jugador, y su embedding es definido por la siguiente estructura:

$$E_{\text{player_state}} = E_{\text{token_type}} + E_{\text{player}} + E_{\text{seat}} + E_{\text{score}} + E_{\text{riichi_status}} + E_{\text{open_hand}} \quad (6)$$

Dónde:

- $E_{\text{token_type}}$ identifica el token como una entidad de tipo `PLAYER_STATE`,
- E_{player} codifica la identidad del jugador,
- E_{seat} representa el viento de asiento del jugador (este/sur/oeste/norte),
- E_{score} codifica los puntos actuales del jugador,
- $E_{\text{riichi_status}}$ indica si el jugador ha declarado riichi,
- $E_{\text{open_hand}}$ indica si el jugador tiene una mano abierta.

7.9 Tensorización y embeddings implementados

La implementación final transforma cada token en seis identificadores discretos: tipo de token, tipo de ficha, índice de copia, jugador, tipo de acción y posición absoluta dentro de la secuencia. Cada identificador dispone de una tabla de embeddings entrenable de dimensión 128. La representación de entrada se obtiene sumando los seis vectores:

$$E = E_{\text{tipo}} + E_{\text{ficha}} + E_{\text{copia}} + E_{\text{jugador}} + E_{\text{acción}} + E_{\text{posición}}. \quad (7)$$

La información tensorizada depende de la categoría del token. Los tokens de mano incorporan el tipo y la copia física de la ficha. Los descartes incorporan tipo de ficha, copia y jugador. Los melds incorporan el jugador propietario y el tipo de llamada, mientras que las acciones candidatas incorporan el tipo de acción y la primera ficha asociada. Todos los tokens reciben, además, su tipo y su posición absoluta en la secuencia.

Algunos metadatos presentes en las clases semánticas no se trasladan todavía a estos identificadores. Entre ellos se encuentran la condición de dora roja, el indicador de *tsumogiri*, la declaración de Riichi asociada a un descarte, las fichas y procedencias completas de los melds, los puntos, el estado de Riichi de cada jugador, el número de honba, los depósitos de Riichi, las fichas restantes y los indicadores de Dora. En particular, los tokens GAME_STATE y PLAYER_STATE conservan su tipo y posición, pero sus demás atributos no se codifican en la entrada de los checkpoints evaluados.

Esta diferencia entre el diseño conceptual y la versión implementada constituye una limitación de la representación actual. Su incorporación requeriría ampliar el tensorizador y volver a entrenar los modelos, por lo que se plantea como trabajo futuro y no se atribuye a los resultados presentados.

7.10 Configuración final del modelo

Los dos modelos comparados utilizan una dimensión de embedding de 128, ocho cabezas de atención, cuatro capas de Transformer Encoder, una dimensión de 512 en la red *feed-forward* y un *dropout* de 0,1. La cabeza de política consiste en una capa lineal que transforma el estado oculto de cada token de acción en un único logit. En total, la arquitectura contiene 867.073 parámetros entrenables.

8 Resultados

En esta sección se presentan los resultados obtenidos por los dos checkpoints conservados, identificados como 100k y 500k. El segundo fue entrenado utilizando los primeros 500.000 estados de la base de datos filtrada. Del checkpoint 100k se conserva el estado del modelo, pero no un registro completo de la ejecución ni los metadatos del optimizador y de la época. Por tanto, sus predicciones pueden reproducirse, aunque no es posible reconstruir íntegramente aquel entrenamiento. Ambos checkpoints comparten la misma arquitectura Transformer y la misma representación de entrada; la denominación 100k se mantiene como identificador del experimento original.

Las métricas registradas durante el entrenamiento permiten analizar el proceso de optimización, pero no constituyen una estimación independiente de la capacidad de generalización. Para obtener una comparación final reproducible, ambos modelos fueron evaluados sobre un mismo conjunto de datos reservado que no había sido utilizado para entrenar ninguno de los dos checkpoints.

8.1 Conjunto de evaluación reservado

La base de datos filtrada contiene un total de 552.563 estados con más de una opción de descarte. Los primeros 500.000 estados fueron utilizados para el entrenamiento del segundo modelo. En consecuencia, los 52.563 estados restantes, correspondientes a las posiciones comprendidas entre 500.000 y 552.562 de la tabla, se reservaron exclusivamente para la evaluación final.

Esta separación garantiza que las filas evaluadas no fueron utilizadas durante el entrenamiento de ninguno de los dos modelos. No obstante, el formato disponible no proporciona un identificador de partida. Por este motivo, la división se realizó según la posición de los estados en la base de datos y no fue posible garantizar que todos los estados pertenecientes a una misma partida quedasen en una única partición. Esta circunstancia debe considerarse una limitación del procedimiento de evaluación.

Los dos checkpoints fueron evaluados sobre exactamente los mismos 52.563 estados, manteniendo sin modificaciones la tokenización, la representación del estado, la arquitectura y el conjunto de acciones utilizado durante el entrenamiento. La inferencia se realizó mediante `torch.inference_mode()`, que evita almacenar la información necesaria para el cálculo de gradientes, reduciendo el consumo de memoria sin alterar las predicciones obtenidas. El procedimiento necesario para volver a generar las tablas y figuras se recoge en el Anexo V, Manual de Reproducibilidad.

8.2 Métricas predictivas

La evaluación convencional utiliza la pérdida de entropía cruzada y las precisiones top-1, top-3 y top-5. La precisión top-1 representa la proporción de estados en los que la acción con mayor probabilidad coincide con el descarte del jugador. Las métricas top-3 y top-5 consideran correcta una predicción cuando la acción observada se encuentra entre las tres o cinco alternativas con mayor puntuación, respectivamente.

Cuadro 19: Resultados sobre el conjunto de evaluación reservado

Modelo	Pérdida	Top-1	Top-3	Top-5	Precisión estratégica
100k	1,3533	55,81%	83,37%	91,25%	88,97%
500k	0,9440	65,64%	92,47%	97,71%	94,01%

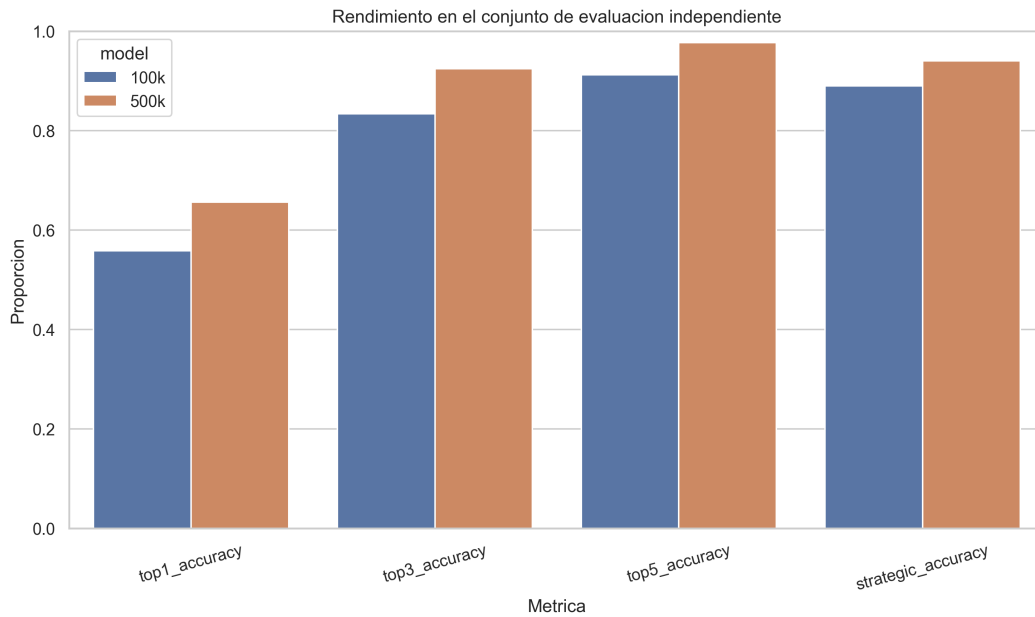


Figura 8: Comparación de las métricas obtenidas por ambos modelos sobre los 52.563 estados reservados para la evaluación final

El aumento del conjunto de entrenamiento produce una mejora consistente en todas las métricas predictivas. La precisión top-1 aumenta del 55,81% al 65,64%, lo que supone una diferencia de 9,83 puntos porcentuales. La precisión top-3 pasa del 83,37% al 92,47%, mientras que la precisión top-5 aumenta del 91,25% al 97,71%. Paralelamente, la pérdida disminuye de 1,3533 a 0,9440.

Estos resultados muestran que el modelo entrenado con 500.000 estados no solo reproduce con mayor frecuencia el descarte observado, sino que también sitúa esa acción entre sus primeras opciones con mayor regularidad. La mejora se observa sobre datos reservados y no únicamente en las métricas registradas durante el entrenamiento.

8.3 Evaluación estratégica

La coincidencia exacta con la acción observada no permite valorar todas las decisiones alternativas que pueden resultar razonables en Mahjong. Por ello, la evaluación predictiva se complementó con una clasificación estratégica basada en el Shanten y el Ukeire.

Las decisiones se clasificaron en las siguientes categorías:

- *Exact match*: el descarte predicho coincide con el descarte observado.
- *Equivalent*: ambos descartes producen el mismo Shanten y el mismo Ukeire.
- *Near equivalent*: ambos descartes mantienen el mismo Shanten y la diferencia absoluta de Ukeire no supera las dos unidades.
- *Better shanten*: el descarte del modelo produce un Shanten menor que el descarte observado.
- *Better ukeire*: ambos descartes mantienen el mismo Shanten, pero el descarte del modelo produce un Ukeire superior en más de dos unidades.

- *Shanten loss*: el descarte del modelo produce un Shanten mayor.
- *Ukeire loss*: ambos descartes mantienen el mismo Shanten, pero el descarte del modelo reduce el Ukeire en más de dos unidades.

La precisión estratégica se define como la proporción de decisiones incluidas en las categorías *exact match*, *equivalent*, *near equivalent*, *better shanten* y *better ukeire*. Esta métrica debe interpretarse como un indicador de eficiencia de la mano basado exclusivamente en Shanten y Ukeire, y no como una medida completa de la calidad estratégica de una jugada.

Cuadro 20: Distribución de categorías estratégicas en el conjunto de evaluación reservado

Categoría	Modelo 100k	Modelo 500k
<i>Exact match</i>	55,81%	65,64%
<i>Equivalent</i>	14,99%	12,29%
<i>Near equivalent</i>	7,67%	6,62%
<i>Better shanten</i>	3,46%	3,24%
<i>Better ukeire</i>	7,03%	6,22%
<i>Shanten loss</i>	5,03%	1,59%
<i>Ukeire loss</i>	6,00%	4,39%

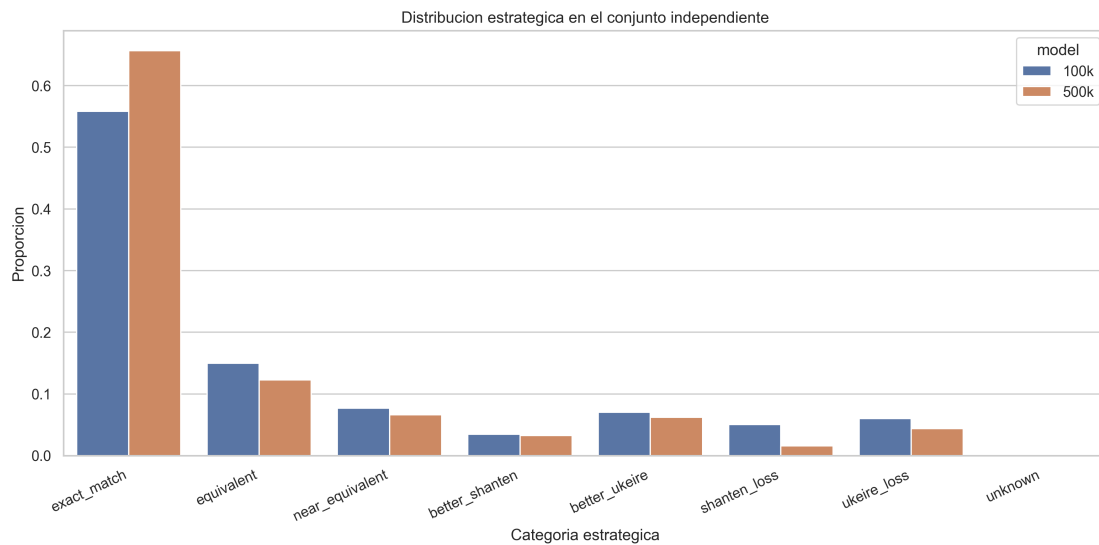


Figura 9: Distribución de las categorías estratégicas de ambos modelos sobre el conjunto de evaluación reservado

La precisión estratégica del checkpoint 100k es del 88,97%, frente al 94,01% alcanzado por el modelo entrenado con 500.000 estados. La diferencia entre ambos modelos es, por tanto, de 5,04 puntos porcentuales.

La mejora más relevante se observa en las decisiones que empeoran el Shanten. Estas representan el 5,03% de las predicciones del primer modelo y solamente el 1,59% de las predicciones del segundo. También disminuyen las decisiones clasificadas como *Ukeire loss*, que pasan del 6,00% al 4,39%.

La diferencia media de Ukeire, calculada como el Ukeire del descarte observado menos el Ukeire del descarte predicho, pasa de $-0,3695$ para el checkpoint 100k a $0,1592$ para el

modelo de 500.000 estados. Un valor negativo indica que, en promedio, el descarte predicho conserva ligeramente más fichas útiles que el descarte observado; un valor positivo indica lo contrario. No obstante, esta media debe interpretarse conjuntamente con el Shanten, puesto que una mejora de Ukeire no compensa necesariamente una pérdida de Shanten.

8.4 Confianza de las predicciones

La confianza media de la acción seleccionada es del 62,83% para el checkpoint 100k y del 68,59% para el modelo entrenado con 500.000 estados. Esta diferencia muestra que el segundo modelo concentra una mayor proporción de probabilidad en su primera opción.

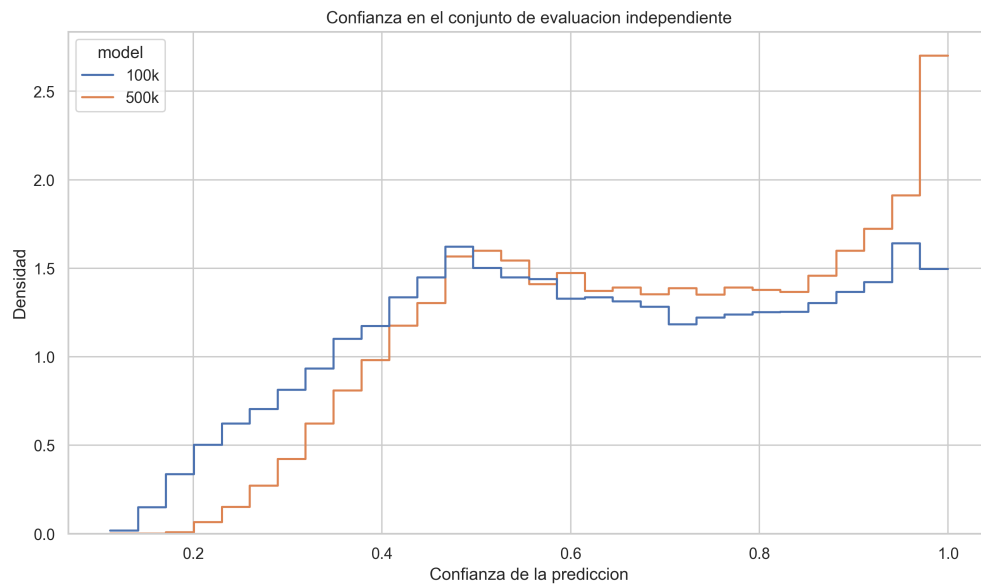


Figura 10: Distribución de la confianza de ambos modelos sobre el conjunto de evaluación reservado

La confianza no debe interpretarse directamente como una probabilidad calibrada de que una decisión sea estratégicamente correcta. En este trabajo representa únicamente la probabilidad asignada por la cabeza de política a la acción seleccionada entre las alternativas disponibles.

8.5 Coste computacional y eficiencia

Para estimar el coste de la propuesta se cronometró una ejecución completa del notebook de evaluación final sobre el equipo utilizado durante el proyecto, equipado con un procesador Intel Core Ultra 7 255HX y una GPU NVIDIA GeForce RTX 5060 Laptop. La inferencia se realizó mediante CUDA, con lotes de 64 estados y los dos checkpoints cargados de forma sucesiva. Los tiempos corresponden a una única ejecución y deben interpretarse como medidas orientativas dependientes del hardware y de la carga del sistema.

Cuadro 21: Tiempo de cálculo de la evaluación sobre los 52.563 estados reservados

Etapas	Unidades procesadas	Tiempo	Rendimiento
Inferencia de ambos checkpoints	105.126 predicciones	167 s	629 predicciones/s
Shanten, Ukeire y categorización	52.563 estados	481 s	109 estados/s

Las dos etapas principales requieren conjuntamente unos 648 segundos, aproximadamente 10 minutos y 48 segundos. La evaluación estratégica concentra el 74,2% de este tiempo, por lo que el principal coste de la evaluación final no procede de la inferencia del Transformer, sino del cálculo repetido de Shanten y Ukeire para los descartes posibles. El modelo contiene 867.073 parámetros entrenables y cada checkpoint ocupa aproximadamente 10,5 MB, lo que permite mantener reducido tanto el tamaño almacenado como el coste de inferencia. No se proporcionan tiempos totales del entrenamiento, ya que no fueron registrados de forma homogénea durante las ejecuciones originales.

8.6 Análisis comparativo

La evaluación sobre el conjunto reservado muestra que el checkpoint de 500k obtiene una pérdida menor y mejores resultados top-1, top-3 y top-5 que el checkpoint de 100k. También alcanza una mayor precisión estratégica y reduce de forma considerable las decisiones que empeoran el Shanten. Esta comparación es compatible con un efecto positivo del mayor volumen de datos, aunque la trazabilidad incompleta del entrenamiento de 100k impide atribuirle la diferencia de forma exclusivamente causal.

La diferencia entre la precisión exacta y la estratégica muestra que una parte de las predicciones que no coinciden con el descarte observado conserva una eficiencia de mano equivalente o superior según las heurísticas consideradas. En el modelo de 100.000 estados, la precisión aumenta del 55,81% al 88,97% al aplicar este criterio. En el modelo de 500.000 estados, aumenta del 65,64% al 94,01%.

Sin embargo, estos porcentajes no demuestran que todas las decisiones clasificadas como estratégicamente correctas sean óptimas. El Shanten y el Ukeire describen principalmente la eficiencia ofensiva de la mano, pero no incorporan factores como el valor esperado, la seguridad defensiva, la situación de puntos, el riesgo frente a jugadores en Riichi o la evolución completa de la partida. Por ello, la precisión estratégica debe entenderse como una herramienta complementaria de análisis y no como una medida absoluta del nivel de juego del modelo.

9 Conclusiones y Líneas de Trabajo Futuro

El objetivo principal de este trabajo ha sido estudiar la aplicación de arquitecturas Transformer al problema de la predicción de descartes en Riichi Mahjong mediante aprendizaje por imitación utilizando partidas reales de Tenhou. Para ello se diseñó una representación propia de las instantáneas disponibles, un proceso de tokenización adaptado al dominio y un modelo basado en un Transformer Encoder capaz de aproximar los descartes observados en el dataset.

Los resultados obtenidos muestran que el modelo es capaz de aprender patrones relevantes para la selección de descartes a partir de los datos disponibles. Sobre un conjunto reservado de 52.563 estados, el checkpoint identificado como 100k alcanzó una precisión exacta del 55,81% y una precisión estratégica del 88,97%. El modelo entrenado con 500.000 muestras mejoró estos resultados hasta el 65,64% y el 94,01%, respectivamente. También aumentó la precisión top-3 del 83,37% al 92,47% y redujo la pérdida de 1,3533 a 0,9440.

Desde el punto de vista estratégico, el incremento del conjunto de entrenamiento redujo las decisiones que empeoran el Shanten del 5,03% al 1,59% y las decisiones que reducen significativamente el Ukeire del 6,00% al 4,39%. Estos resultados, obtenidos sobre estados no utilizados durante el entrenamiento, proporcionan una evidencia más sólida de la mejora introducida por el mayor volumen de datos. No obstante, debido a la ausencia de identificadores de partida, la separación se realizó por posición en la base de datos y no pudo garantizarse la independencia entre partidas.

No obstante, la principal aportación de este trabajo no reside únicamente en el desarrollo del modelo, sino en la metodología propuesta para su evaluación. Las métricas de clasificación, como la exactitud (*accuracy*), consideran correcta únicamente la reproducción exacta de la acción observada. En un juego como el Riichi Mahjong, donde una misma situación puede admitir varias decisiones con una eficiencia de mano similar, este criterio no describe por sí solo todas las diferencias entre los descartes posibles.

Con el objetivo de complementar esta información, en este trabajo se propuso una validación estratégica basada en Shanten y Ukeire. La metodología distingue las decisiones que empeoran estas heurísticas de aquellas que mantienen o mejoran la eficiencia ofensiva de la mano. Gracias a este enfoque fue posible observar que una parte de las predicciones que no coinciden con la acción registrada resultan equivalentes o superiores según los criterios concretos de Shanten y Ukeire empleados.

Sin embargo, estas métricas no deben interpretarse como una medida absoluta de la inteligencia del modelo. Una precisión estratégica elevada no implica necesariamente que el sistema haya comprendido todos los aspectos del juego, sino únicamente aquellos representados por las heurísticas utilizadas durante la evaluación. En consecuencia, estas métricas deben entenderse como una herramienta de análisis que permite describir con mayor precisión el comportamiento del modelo, y no como una medida definitiva de su nivel de juego.

10 Líneas de Trabajo Futuro

Aunque los resultados obtenidos son prometedores, existen numerosas líneas de trabajo que permitirían ampliar y mejorar tanto el modelo desarrollado como la metodología de evaluación propuesta.

En primer lugar, una posible línea de investigación consiste en ampliar las métricas estratégicas utilizadas durante la evaluación. En este trabajo únicamente se consideraron el

Shanten y el Ukeire como indicadores de calidad estratégica. Sin embargo, el Riichi Mahjong incorpora numerosas heurísticas adicionales relacionadas con el juego ofensivo y defensivo, como el Suji, Ura-Suji, Kabe, Genbutsu o el análisis del peligro de descarte frente a jugadores en Riichi. La incorporación de este tipo de criterios permitiría construir una evaluación mucho más completa del comportamiento estratégico del modelo.

De forma similar, la propia clasificación estratégica podría extenderse para analizar otros aspectos de la toma de decisiones, permitiendo estudiar no solamente si un descarte mejora la eficiencia de la mano, sino también si representa una decisión defensivamente adecuada o consistente con estrategias de mayor nivel empleadas por jugadores expertos.

Desde el punto de vista del modelo, otra línea de trabajo consiste en explorar arquitecturas más avanzadas. Aunque en este proyecto se ha empleado un Transformer Encoder relativamente sencillo para facilitar el análisis del comportamiento del sistema, existen variantes modernas como Decision Transformer o arquitecturas específicas para aprendizaje secuencial que podrían aprovechar mejor la naturaleza temporal de las partidas de Mahjong.

Asimismo, este trabajo se ha centrado exclusivamente en la decisión de descarte, que constituye la acción más frecuente durante una partida. Como continuación natural del proyecto, sería interesante ampliar el espacio de acciones para incluir decisiones como Chi, Pon, Kan, Riichi o incluso Tsumo y Ron. Esto permitiría construir un agente capaz de participar de manera completa en una partida real de Riichi Mahjong.

Finalmente, otra línea especialmente interesante consiste en utilizar el modelo obtenido mediante aprendizaje por imitación como punto de partida para un proceso posterior de aprendizaje por refuerzo. En este contexto, las métricas estratégicas desarrolladas en este trabajo podrían resultar útiles para el diseño de funciones de recompensa más informativas que la simple victoria o derrota al final de la partida. Incorporar información relacionada con el Shanten, el Ukeire o futuras heurísticas estratégicas permitiría proporcionar una señal de aprendizaje más densa durante el entrenamiento, facilitando potencialmente la convergencia del agente y favoreciendo la adquisición de comportamientos estratégicamente sólidos.

11 Glosario

Este glosario recoge los términos japoneses e ingleses empleados con mayor frecuencia. En los términos japoneses se incluye la grafía habitual en japonés y su romanización.

11.1 Términos de Riichi Mahjong

Riichi Mahjong — リーチ麻雀 (*rīchi mājan*)

Variante japonesa del Mahjong empleada en este trabajo.

Riichi — 立直／リーチ (*rīchi*)

Declaración realizada cuando una mano cerrada queda en Tenpai, acompañada del depósito de 1.000 puntos.

Shanten — 向聴数／シャンテン数 (*shanten-sū*)

Distancia estructural de una mano respecto de Tenpai. Un valor menor indica que la mano está más cerca de completarse.

Tenpai — 聴牌／テンパイ (*tenpai*)

Estado en el que falta una sola ficha válida para completar la mano.

Ukeire — 受け入れ／ウケイレ (*ukeire*)

Número de copias todavía disponibles de las fichas que mejoran el Shanten de la mano.

Dora — ドラ (*dora*) Ficha que aporta valor adicional a una mano ganadora, determinada a partir de un indicador.

Aka Dora — 赤ドラ (*aka dora*)

Cinco rojo que cuenta como Dora.

Tsumo — 自摸／ツモ (*tsumo*)

Robo de una ficha; también, victoria obtenida con una ficha robada por el propio jugador.

Tsumogiri — ツモ切り (*tsumogiri*)

Descarte inmediato de la misma ficha que se acaba de robar.

Ron — ロン (*ron*) Victoria conseguida utilizando la ficha descartada por otro jugador.

Chi — チー (*chī*) Llamada que forma una secuencia con el descarte del jugador situado a la izquierda.

Pon — ポン (*pon*) Llamada que forma un trío con una ficha descartada por otro jugador.

Kan — 槓／カン (*kan*)

Grupo de cuatro fichas idénticas. Puede ser cerrado, abierto o ampliado.

Ankan — 暗槓 (*ankan*)

Kan cerrado formado con cuatro fichas propias.

Daiminkan — 大明槓 (daiminkan)

Kan abierto completado mediante el descarte de otro jugador.

Shouminkan o Kakan — 小明槓／加槓 (shōminkan/kakan)

Ampliación de un Pon abierto al añadir la cuarta ficha.

Mentsu — 面子 (mentsu)

Grupo que forma parte de una mano: secuencia, trío o Kan.

Fūro — 副露 (fūro)

Grupo expuesto mediante una llamada a una ficha de otro jugador.

Yaku — 役 (yaku)

Patrón o condición que concede valor y permite que una mano pueda ganar.

Furiten — 振聴／フリテン (furiten)

Situación que impide ganar por Ron sobre las esperas afectadas, aunque todavía puede ser posible ganar por Tsumo.

Honba — 本場 (honba)

Contador de rondas consecutivas que incrementa los pagos de la mano.

Kyūshukyūhai — 九種九牌 (kyūshukyūhai)

Opción de abortar la mano inicial cuando se cumplen las condiciones de fichas terminales y de honor distintas.

Chankan — 槍槓 (chankan)

Victoria al reclamar la ficha con la que otro jugador intenta ampliar un Kan.

Suji — 筋 (suji)

Heurística defensiva basada en las relaciones entre esperas de secuencia.

Kabe — 壁 (kabe)

Heurística defensiva que aprovecha las fichas visibles para descartar ciertas esperas posibles.

Genbutsu — 現物 (genbutsu)

Ficha ya descartada por un jugador en Riichi y, por ello, segura contra su Ron mientras se mantenga esa situación.

Ura-suji — 裏筋 (ura-suji)

Patrón defensivo relacionado con posibles esperas situadas alrededor de un descarte.

11.2 Términos técnicos en inglés

Accuracy

Exactitud: proporción de predicciones que cumplen el criterio considerado correcto.

Batch

Lote de muestras procesadas conjuntamente durante el entrenamiento o la evaluación.

Checkpoint	Fichero que conserva los parámetros de un modelo en un momento determinado.
Dataset	Conjunto de datos utilizado para el entrenamiento o la evaluación.
Dropout	Técnica de regularización que desactiva aleatoriamente parte de las activaciones durante el entrenamiento.
Embedding	Representación vectorial aprendida de un identificador discreto.
Encoder	Componente que transforma una secuencia de entrada en representaciones contextuales.
Feed-forward	Red neuronal aplicada de forma independiente a cada posición dentro de una capa Transformer.
Holdout	Conjunto reservado que no se utiliza para entrenar el modelo y se emplea en la evaluación final.
Logit	Puntuación no normalizada producida por el modelo antes de aplicar una distribución de probabilidad.
Mask	Máscara que indica qué posiciones deben atenderse o ignorarse durante el procesamiento.
Policy head	Capa de salida que asigna una puntuación a cada acción candidata.
Self-attention	Mecanismo que relaciona cada elemento de una secuencia con los demás elementos de esa misma secuencia.
Softmax	Función que transforma un conjunto de logits en una distribución de probabilidad.
Token	Unidad discreta utilizada para representar una parte de la entrada del modelo.
Top-k accuracy	Proporción de ejemplos en los que la acción observada aparece entre las k predicciones con mayor puntuación.
Transformer	Arquitectura neuronal basada principalmente en mecanismos de atención.

Referencias

- [1] D. Chiba, *Riichi Book 1: The Ultimate Guide to the Japanese Game Taking the World by Storm*. Lulu.com, 2015, ISBN: 978-1329626478.
- [2] J. Devlin, M. Chang, K. Lee y K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *CoRR*, vol. abs/1810.04805, 2018. arXiv: 1810 . 04805. dirección: [http : / / arxiv . org/abs/1810.04805](http://arxiv.org/abs/1810.04805)
- [3] DWANGO Co., Ltd., *Mahjong AI NAGA*, [https : / / dmv . nico / ja / articles / mahjong_ai_naga/](https://dmv.nico/ja/articles/mahjong_ai_naga/), Commercial Riichi Mahjong AI. Accessed: 2026-07-18, 2021.
- [4] Equip, *Mortal Documentation*, <https://mortal.ekyu.moe>, Documentation of the Mortal Riichi Mahjong AI. Accessed: 2026-07-18, 2022.
- [5] Equip, *Mortal: A Fast and Strong AI for Riichi Mahjong*, [https : / / github . com / Equip-chan/Mortal](https://github.com/Equip-chan/Mortal), Open-source Riichi Mahjong AI based on deep reinforcement learning. Accessed: 2026-07-18, 2022.
- [6] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. MIT Press, 2016. dirección: <https://www.deeplearningbook.org/>
- [7] A. Hussein et al., “Imitation Learning: A Survey of Learning Methods”, *ACM Computing Surveys*, 2017.
- [8] Japanese Mahjong Wiki. “List of yaku”, visitado 24 de ago. de 2026. dirección: [https : //riichi.wiki/List_of_yaku](https://riichi.wiki/List_of_yaku)
- [9] Japanese Mahjong Wiki. “Rules overview”, visitado 9 de jun. de 2026. dirección: https://riichi.wiki/Rules_overview
- [10] J. Li et al., “Suphx: Mastering Mahjong with Deep Reinforcement Learning”, *arXiv preprint arXiv:2003.13590*, 2020.
- [11] X. Li, B. Liu, Z. Wei, Z. Wang y L. Wu, “Tjong: A Transformer-Based Mahjong AI via Hierarchical Decision-Making and Fan Backward”, *CAAI Transactions on Intelligence Technology*, vol. 9, n° 4, págs. 982-995, 2024. doi: 10.1049/cit2.12298 dirección: <https://doi.org/10.1049/cit2.12298>
- [12] A. Vaswani et al., “Attention Is All You Need”, *NeurIPS*, 2017.